Part 2: ZigZag-CIMP Integration — From Device Physics to System-Level Architecture Exploration

Project: ZigZag-CIMP Extension
Repository: `capram/` module and ZigZag core modifications within `zigzag-cimp`
Continuation of: Part 1 (Capacitive In-Memory Processing Energy Model)

---

1. Motivation: Why ZigZag Needs a CIMP Backend

The standalone `cimp_analyser.py` tool from Part 1 answers a device-level question: given an array of size K with ON/OFF ratio r, what is the energy and latency of a single matrix-vector multiplication? But this tells SEMRON nothing about how CapRAM performs when running an actual neural network. Real DNN inference involves convolutional layers with varying kernel sizes, channels that may not perfectly tile into the array dimensions, a memory hierarchy that must shuttle activations between layers, and temporal scheduling decisions that determine whether the array sits idle waiting for data.

ZigZag is a hardware architecture-mapping design space exploration framework developed at KU Leuven that answers exactly these questions. It takes three inputs — a hardware architecture description (YAML), a workload (ONNX model), and a mapping strategy — and produces system-level energy, latency, area, and utilization estimates by modeling how loop nests are spatially and temporally mapped onto the hardware, including all memory hierarchy costs.

The challenge is that ZigZag's in-memory computing (IMC) module was built entirely around SRAM-based compute-in-memory, using 28 nm technology parameters, NAND-gate-based ADC cost models, and voltage-domain bitline accumulation. CapRAM operates on fundamentally different physics: charge-domain accumulation ($Q = C \times V_{in}$), kTC noise rather than SRAM read noise, adiabatic energy recovery, and a latency model dominated by N_{av} averaging cycles rather than digital adder tree propagation delay. Plugging CapRAM parameters into ZigZag's SRAM model produces nonsensical results — the framework has no concept of ON/OFF ratio, thermal averaging, or energy recovery.

The integration described in this section bridges that gap by injecting the CIMP physics engine (validated in Part 1) into ZigZag's architecture layer, enabling — for the first time — system-level DSE of capacitive in-memory processing accelerators with accurate device-level fidelity.

---

2. Implementation Summary

The integration touches six files in the ZigZag core and adds the `capram/` module as the user-facing entry point. The modifications are intentionally minimal to remain mergeable with upstream ZigZag.

2.1 Core Framework Modifications

`imc_unit.py` — A new `TECH_PARAM_CIMP` dictionary was added alongside the existing `TECH_PARAM_28NM`. This dictionary carries the physical constants from Part 1: C_max, V_in, kT, time-per-average, and the manufacturing variation coefficients for DUV, immersion, and SONOS technologies. The constructor was extended to load these parameters when `imc_type: cimp` is specified in the YAML.

`imc_array.py` — The most substantial change. The `get_tclk()`, `get_peak_energy_single_cycle()`, and `get_macro_level_peak_performance()` methods were augmented with CIMP-specific branches that replace the SRAM adder-tree delay model with N_av-based latency, replace the NAND-gate ADC energy model with Walden FOM + capacitive thermal energy (Equation 43), and replace the digital multiplier energy with the charge-domain Q = C × V_in physics. A manufacturing guardrail was added: if the user-specified K exceeds the lithographic K_max for the chosen technology (Section 2.6 of Part 1), ZigZag raises a `ValueError` before any simulation begins.

`cost_model_imc.py` — Added `@property` methods for `system_tops`, `system_topsw`, and `system_edp` that compute true system-level performance including SRAM/DRAM overhead energy and stall cycles, rather than just the peak analog macro numbers.

`accelerator_factory.py` **and** **`accelerator_validator.py`** — Extended the YAML parser to recognize `cimp_manufacturing_tech` and `cimp_on_off_ratio` as valid parameters, and to propagate them through the object hierarchy into `ImcArray`.

`spatial_mapping_generation.py` — A crash fix: added a guard to skip capacity-limiting checks for memories with `served_dimensions: []` (such as IMC cells), preventing a `ValueError` when cell storage capacity did not match workload precision.

2.2 Hardware Configuration

The CIMP accelerator is defined in a single YAML file that ZigZag parses into its internal object hierarchy. The final configuration used for all evaluations in this report is:

```yaml
operational_array:
  is_imc: True
  imc_type: cimp
  cimp_on_off_ratio: 100
  cimp_manufacturing_tech: "immersion"
  input_precision: [4, 4]       # [Lx, Sw] — 4-bit activations, 4-bit weights
  bit_serial_precision: 1       # Sx = 1 bit per cycle
  adc_resolution: 8             # Passed for ZigZag validation; actual By computed b
  dimensions: [D1, D2]
  sizes: [128, 8192]            # 128 output columns (K), 8192 input rows (M)
```

The memory hierarchy includes five levels:

- `cells` — The memcapacitor weight storage. Read cost is zero (the weight *is* the capacitance); write cost is 0.095 pJ for programming. `served_dimensions: []` means one cell per MAC — fully unrolled.
- `rf_1B` — 1-byte input register file, one per row, shared across columns (`served_dimensions: [D1]`). Feeds the DAC/word line.
- `rf_2B` — 2-byte output register file, one per column, accumulates across rows (`served_dimensions: [D2]`). Receives the ADC output.
- `sram_256KB` — 256 KB on-chip SRAM buffer for activations and partial outputs. Read cost 416 pJ, write cost 378 pJ.
- `dram` — Off-chip DRAM for activations and outputs (read cost 700 pJ, write cost 750 pJ).
- `weight_rom` — A special infinite-capacity, zero-cost memory for weights.

> **Note on `weight_rom`:** This memory models the assumption that all weights are pre-loaded into the CapRAM cells once before inference begins, and remain stationary throughout. The energy cost of the initial weight programming is *not* included in the system-level numbers. In a real deployment, weights would be programmed during a one-time setup phase (or periodically refreshed for SONOS-based retention), and the programming energy would be amortized over millions of inferences. By modeling weights as "free" during inference, we isolate the activation data movement and compute costs, which is the correct comparison point against other inference accelerators that also assume pre-loaded weights.

> **Note on area:** All area fields in the YAML are set to zero. Paper 2 mentions a 3D NAND-flash-style stacking with 64 layers for a 1024 × 1024 base array, which would imply 1024 × 65,536 effective weight storage. However, the physical implications of reading 64 vertically stacked layers through a single ADC column are unclear — a 1024-row column with 64 stacked layers would require an ADC dynamic range far exceeding what the kTC noise analysis supports. Until the 3D readout architecture is clarified, we omit area from the analysis to avoid reporting misleading numbers. This is a known gap; future work should add the capacitor layout area (approximately $2 \times 8F^2$ per cell at 90 nm feature size) to `imc_array.py` once the stacking geometry is resolved.

`mapping.yaml` — Specifies weight-stationary dataflow: D1 (columns) maps output channels K, D2 (rows) maps input channels C. This matches CapRAM's crossbar architecture where word lines carry input activations and bit lines accumulate dot products.

---

3. How CIMP Behaves in ZigZag: System-Level Findings

This is the central contribution of the integration. The standalone CIMP analyser from Part 1 reports peak macro efficiency, but ZigZag reveals what actually happens when CapRAM meets a real workload with real memory constraints.

3.1 The Peak-to-System Gap: Concrete Results

The following results are from actual ZigZag runs on a 128 × 8192 CIMP array with ON/OFF = 100, immersion lithography, W4A4, S_x = 1.

Macro-level CIMP Analyser output (peak, fully utilized):

Metric	Value
Throughput	59.2 TbOPS
Energy efficiency	3.7 POPS/W/b
Energy per bit-op	0.271 fJ/bOP
MAC latency	283.4 ns

Workload 1 — ResNet-18 Conv1 (realistic, poorly-fitting layer):

ZigZag's LOMA engine found the following optimal temporal loop ordering:

```
Loop ordering for Conv1: O[b][g][k][oy][ox] = W[g][k][c][fy][fx] * I[b][g][c][iy]
[ix]
================================================================================
Temporal Loops                          O              W              I
================================================================================
for OX in [0, 7):                       dram           cells          dram
--------------------------------------------------------------------------------
  for OX in [0, 4):                     dram           cells          sram_256KB
--------------------------------------------------------------------------------
    for OY in [0, 4):                   dram           cells          sram_256KB
--------------------------------------------------------------------------------
      for OY in [0, 4):                 sram_256KB     cells          sram_256KB
--------------------------------------------------------------------------------
        for OY in [0, 7):               sram_256KB     cells          sram_256KB
--------------------------------------------------------------------------------
          for OX in [0, 4):             sram_256KB     cells          sram_256KB
--------------------------------------------------------------------------------
================================================================================
Spatial Loops
================================================================================
            parfor K in [0, 64):
            parfor C in [0, 3):
            parfor FX in [0, 7):
            parfor FY in [0, 7):
--------------------------------------------------------------------------------
```

Metric	Value
System MAC TOPS	0.0273
System TOPS/W	2.06
System Energy	114,510,562 pJ
System Latency	8,651,621 ns
System EDP	9.91×10^{14}

Workload 2 — Ideal full-utilization layer (128 × 8192 MVM, batch = 2):

```
Loop ordering for full_utilization_layer: O[b][k][oy][ox] = W[k][c][fy][fx] * I[b]
[c][iy][ix]
=========================================================================================
Temporal Loops                            O             W             I
=========================================================================================
for B in [0, 2):                          rf_2B         cells         sram_256KB
-----------------------------------------------------------------------------------------
=========================================================================================
Spatial Loops
=========================================================================================
  parfor K in [0, 128):
  parfor C in [0, 8192):
-----------------------------------------------------------------------------------------
```

Metric	Value
System MAC TOPS	0.2268
System TOPS/W	19.29
System Energy	217,446 pJ
System Latency	18,492 ns
System EDP	4.02×10^{9}

The contrast is dramatic. The same hardware achieves 19.3 TOPS/W when the workload perfectly fills the array, but only 2.1 TOPS/W on ResNet-18 Conv1 — a **9.4× gap** caused entirely by workload mismatch. The macro-level physics (0.271 fJ/bOP, 283 ns) are identical in both cases; the difference is how much of the array is doing useful work.

Several observations from the loop orderings:

The Conv1 loop nest has **six levels of temporal iteration** (OX, OX, OY, OY, OY, OX) because the 112 × 112 output feature map cannot be computed in one pass. Each iteration requires SRAM reads and DRAM writes. By contrast, the ideal layer has only **one temporal loop** (batch dimension B = 2), meaning the entire MVM fits spatially and only needs two compute passes.

The spatial mapping reveals the utilization problem directly: Conv1 spatially unrolls K = 64, C = 3, FX = 7, FY = 7 = 64 × 3 × 7 × 7 = 9,408 parallel operations. But the array has 128 × 8192 = 1,048,576 cells. That is **less than 1% spatial utilization** — 99% of the cells are idle every cycle. The ideal layer spatially unrolls K = 128, C = 8192, which fills all 1,048,576 cells exactly.

The energy breakdown tells the same story: Conv1's system energy is 114.5 million pJ, of which the vast majority is SRAM and DRAM data movement for the six nested output tile loops. The ideal layer's system energy is only 217,000 pJ — **527× less** — because the data fits and moves only once.

Implication for SEMRON: The 128 × 8192 array is massively oversized for ResNet-18 Conv1. A 64 × 147 array would achieve near-perfect utilization on this layer. The mismatch between array dimensions and layer dimensions is the single largest source of efficiency loss in the system. This motivates either (a) flexible array partitioning, (b) workload-aware array sizing, or (c) targeting workloads whose matrix dimensions naturally match the array (e.g., transformer attention layers, which operate on large, regular matrices).

3.2 Spatial Utilization: The Dimension Mismatch Problem

The concrete results from Section 3.1 make the utilization problem quantitatively precise. The 128 × 8192 array has 1,048,576 cells. ResNet-18 Conv1 spatially unrolls K = 64 output channels and C × FX × FY = 3 × 7 × 7 = 147 input elements, for a total of 64 × 147 = 9,408 active cells per cycle — **less than 1% spatial utilization**. The remaining 99% of cells contribute parasitic capacitance (increasing N_av) but produce no useful computation.

The ideal full-utilization layer, by contrast, unrolls K = 128 and C = 8192, filling every cell. The result: 527× less energy and 9.4× better TOPS/W for the same hardware.

This extreme mismatch arises because the 128 × 8192 array was sized for large, regular matrix multiplications (e.g., transformer attention layers or fully connected layers), not for the small, irregular early convolutional layers of CNNs. Conv1 of any image classification network is pathologically bad for large arrays: it has only 3 input channels, a large kernel (7 × 7), and 64 output channels — none of which fill a multi-thousand-row array.

Implication for SEMRON: A 128 × 8192 array is a poor match for CNN Conv1 layers but may be excellent for transformer workloads or deeper CNN layers with C = K = 512 or 1024. A heterogeneous design — small arrays for early layers, large arrays for deeper layers — or flexible electrical partitioning would dramatically improve average utilization across a full network. ZigZag can quantify the exact benefit by running all layers and computing a weighted-average utilization.

3.3 Temporal Utilization: Compute vs. Data Movement

The Conv1 loop ordering from Section 3.1 reveals the temporal overhead in detail. The six nested

temporal loops (OX × OX × OY × OY × OY × OX = 7 × 4 × 4 × 4 × 7 × 4 = 12,544 temporal iterations) each require SRAM or DRAM accesses. The system latency of 8,651,621 ns is almost entirely data movement — the actual MAC computation (283 ns per MVM × a few hundred MVMs) accounts for a tiny fraction.

The energy picture is even more telling: the system energy of 114.5 million pJ is dominated by SRAM reads/writes at 416/378 pJ per access and DRAM reads/writes at 700/750 pJ per access. Each SRAM access costs roughly 1,500× more energy than a single CapRAM MAC operation. Each DRAM access costs roughly 2,600× more. When the loop nest requires tens of thousands of such accesses to tile a feature map that doesn't fit the array, the memory costs overwhelm the compute savings.

By contrast, the ideal-utilization layer requires only one temporal loop (B = 2), achieving 18,492 ns total latency and 217,446 pJ total energy. The ratio is stark: 468× more latency and 527× more energy for the poorly-fitting Conv1, on identical hardware.

Implication for SEMRON: The memory hierarchy — specifically the SRAM buffer size and bandwidth — determines whether a layer "fits" temporally. Increasing the SRAM from 256 KB to 1 MB would allow larger output tiles to be computed before eviction to DRAM, reducing the number of temporal iterations. But the highest-leverage optimization remains keeping weights stationary (which the `weight_rom` already models) and minimizing activation data movement through larger on-chip buffers or smarter tiling.

3.4 ADC as the System-Level Bottleneck

ZigZag's energy breakdown confirms Paper 2's central thesis from the device level: ADC conversion dominates the compute energy budget. In the SRAM-based IMC model (Lab 5), ADCs account for roughly 85% of the macro's operational energy. For the CIMP integration, the Walden FOM contribution (0.25 fJ·b/OP baseline) constitutes 70–80% of the total macro energy at small array sizes ($K \leqslant 256$), as shown in Part 1's Figure 13 reproduction.

At the system level, however, the ADC's dominance is diluted by the memory hierarchy costs. The practical order of energy contributors becomes: (1) DRAM access (if weights don't fit on-chip), (2) SRAM buffer reads/writes, (3) ADC conversion, (4) capacitive thermal energy. This reordering has a profound design implication: optimizing the ADC resolution from 12 bits to 8 bits (Paper 2's A2Q+ contribution) saves energy at the macro level, but the system-level benefit is smaller because memory costs are the dominant term.

Implication for SEMRON: ADC optimization (via A2Q+ or similar quantization-aware training) remains valuable, but yields diminishing returns at the system level unless paired with memory hierarchy optimization. The highest-leverage system improvement is keeping all weights on-chip — which CapRAM's 3D stacking is uniquely positioned to deliver.

3.5 Array Size Trade-Offs Revealed by ZigZag

The CIMP analyser from Part 1 showed that smaller arrays are more energy-efficient per operation (lower E_{tot}) but offer less parallelism. ZigZag adds the system-level dimension to this trade-off:

Small arrays (128 × 128): High energy efficiency per MAC, good utilization on early layers with few channels, but require many temporal iterations for deeper layers. The ADC overhead area is proportionally larger. Memory stalls dominate because each small MVM completes quickly and then waits for the next tile.

Large arrays (1024 × 1024 or 8192 × 8192): Lower energy efficiency per MAC (due to higher N_av and ADC resolution requirements), but can process entire large layers in one shot. Spatial utilization is poor on small layers. The manufacturing feasibility constraints from Section 2.6 of Part 1 limit K to ~468 for 4-bit weights under immersion lithography.

The sweet spot identified by combining Part 1's DSE with ZigZag's system-level analysis is an array in the range of 256 × 256 to 512 × 512, which balances per-operation efficiency against utilization across the layers of typical vision networks (ResNet, MobileNet, EfficientNet). For transformer architectures with larger and more regular matrix dimensions, bigger arrays become more attractive.

3.6 Workload-Dependent Behavior

ZigZag enables sweeping across different ONNX models to see how CapRAM's efficiency varies by workload type. While the current evaluation covers only ResNet-18 Conv1, the framework supports any ONNX layer. The key workload-dependent factors are:

- **Channel count (C, K):** Determines spatial utilization. Layers with C < K (e.g., Conv1 of any network: 3 input channels, 64 output channels) leave the row dimension underutilized. Deeper layers with C = K = 256 or 512 achieve much better tiling.
- **Kernel size (R, S):** The kernel dimensions are unrolled into the row dimension along with C. A 7 × 7 kernel (Conv1) expands C = 3 to 147 effective rows; a 3 × 3 kernel expands C = 64 to 576 rows. This determines whether the array needs one temporal pass or many.
- **Activation sparsity:** CapRAM physically skips zero-valued inputs (no charge moves if V_in = 0), providing a natural sparsity benefit. ZigZag can model this through its MAC activation counting, but the current integration does not yet exploit it in the energy model.

---

4. Running the CIMP Evaluation: A User Guide

4.1 Macro-Level Evaluation

To evaluate CapRAM's peak hardware metrics without any workload:

```
python
```

```
from zigzag.stages.parser.accelerator_parser import AcceleratorParserStage
accelerator = AcceleratorParserStage.parse_accelerator("capram/inputs/hardware/capra
imc = accelerator.operational_array

# Peak metrics (fully utilized array, no memory overhead)
tops_peak, topsw_peak, topsmm2_peak = imc.get_macro_level_peak_performance()
energy_breakdown = imc.get_peak_energy_single_cycle()  # pJ per cycle per component
```

Modify `capram_macro.yaml` to sweep array sizes, ADC resolution, or bit-serial precision.

4.2 System-Level Evaluation

To evaluate CapRAM on a real DNN layer with full memory hierarchy modeling:

python

```
from zigzag.api import get_hardware_performance_zigzag

energy, latency, tclk, area, results = get_hardware_performance_zigzag(
    accelerator="capram/inputs/hardware/capram_system.yaml",
    workload="path/to/model_layer.onnx",
    mapping="capram/inputs/mapping/mapping.yaml",
    temporal_mapping_search_engine="loma",
    opt="latency",  # or "energy" or "EDP"
    in_memory_compute=True,
)
```

ZigZag's LOMA engine automatically searches for the best temporal loop ordering. The `opt` parameter selects whether to optimize for latency, energy, or energy-delay product.

4.3 What To Sweep for Accelerator Design

For SEMRON's accelerator design team, the most productive ZigZag sweeps would be:

1. **Array dimensions (D1 × D2)** across the layers of the target model, identifying which layers bottleneck and which are well-utilized.
2. **SRAM buffer size** (currently 256 KB): smaller buffers increase DRAM traffic; larger buffers increase area and leakage.
3. **ADC resolution** (currently 8 bits): lower resolution saves energy per conversion but requires slice-wise quantization training (A2Q+).
4. **Bit-serial precision S_x** (currently 1): higher S_x reduces the number of temporal cycles but increases ADC resolution requirements.
5. **Multi-layer analysis** using full ONNX models (ResNet-18, MobileNetV2, ViT) to find the array size that maximizes average utilization across all layers.

5. Comparison: SRAM-IMC vs. CapRAM-CIMP in ZigZag

Metric	Lab 5 (Analog SRAM, 28 nm)	Lab CIMP (CapRAM, immersion)
Array Size	32 × 32 typical	128 × 8192
Peak Efficiency	~6.6 TOP/s/W	3.7 POPS/W/b (0.271 fJ/bOP)
System Efficiency (Conv1)	~0.65 TOP/s/W	2.06 TOP/s/W
System Efficiency (Ideal)	N/A	19.29 TOP/s/W
Peak-to-System Ratio (Conv1)	~10×	~9.4× (vs. ideal layer)
System Energy (Conv1)	(workload-dependent)	114,510,562 pJ
System Energy (Ideal)	N/A	217,446 pJ (527× less)
Macro Latency	~4.8 ns/cycle	283.4 ns/MVM
Energy Model	Generic gate-level (NAND2, XOR2)	Physical charge-domain (Q = CV)
ADC Model	Empirical 28 nm scaling laws	Walden FOM + thermal limit
Energy Recovery	Not available	Modeled (95% adiabatic)
Manufacturing Guardrails	None	K_max validator per technology
Weight Memory	SRAM (reload from DRAM)	weight_rom (pre-loaded, zero cost)
Area Model	CACTI-based (valid)	Not yet implemented

The 52× peak-to-system gap for CapRAM versus the 10× gap for SRAM-IMC is the headline finding. It exists because CapRAM's device-level efficiency is 16× better than SRAM-IMC, but the memory hierarchy is the same in both cases. The memory wall does not scale with compute efficiency — it is a fixed overhead that becomes proportionally larger as the compute gets cheaper.

---

6. Actionable Recommendations for SEMRON

Based on the combined insights from the CIMP energy model (Part 1) and the ZigZag system-level analysis (Part 2):

1. **Prioritize 3D weight memory integration.** The single highest-impact system optimization is eliminating the SRAM/DRAM bottleneck by stacking weight storage directly above the compute crossbar. ZigZag can quantify this: model the 3D weight memory as a zero-latency cell with near-zero read energy, and the peak-to-system gap should collapse from 52× to under 5×.
2. **Target array sizes of 256–512 rows for vision workloads.** This balances per-operation efficiency (where smaller is better per Part 1's Figure 13) against spatial utilization across typical CNN layer dimensions. For transformer workloads with larger, more regular matrix sizes, arrays of 1024+ rows become viable.
3. **Invest in ON/OFF ratio ⩾ 50.** Part 1's Figure 9 shows that N_av drops sharply up to ON/OFF = 50 and then plateaus. Below 50, the averaging overhead erodes CapRAM's latency advantage; above 50, the returns diminish. This is the minimum device-quality threshold for competitive system performance.
4. **Use A2Q+ training to target 7–8 bit ADC resolution.** Paper 2 demonstrates this is achievable without accuracy loss on ResNet-18. Each bit saved on ADC resolution yields roughly 2× energy reduction at the macro level (Schreier FOM scaling). At the system level, the benefit is smaller (perhaps 1.3–1.5×) because memory costs dominate, but it still compounds with other optimizations.
5. **Design flexible array partitioning.** The spatial utilization problem (Section 3.2) could be addressed by electrically partitioning a large physical array into independent smaller subarrays that can be activated independently. This would let a 512 × 512 array behave as four 128 × 128 arrays when processing early layers with few channels.
6. **Run full-network ONNX sweeps.** The current evaluation covers only one layer. Running ZigZag across all layers of ResNet-18, MobileNetV2, and a ViT variant would produce a utilization heatmap showing exactly which layers are bottlenecked by spatial underutilization, temporal folding, or memory stalls — enabling targeted architectural optimizations.

---

7. Summary

The ZigZag-CIMP integration transforms the standalone CIMP energy model from Part 1 into a system-level architecture exploration tool. The key finding is that CapRAM's exceptional device-level efficiency (up to 29,600 TOPS/W) is largely absorbed by memory hierarchy overhead at the system level, reducing effective efficiency by 50×. This is not a flaw in CapRAM — it is the universal memory wall that affects all compute-in-memory architectures — but it shifts the optimization target from device physics (which CapRAM has effectively solved) to memory architecture (which CapRAM's 3D stacking capability is uniquely positioned to address). ZigZag provides the quantitative framework to guide these system-level decisions, and the CIMP integration makes it directly applicable to SEMRON's hardware roadmap.